Theory - Neural Network Expressivity (20pt)

Part1:

Proof (by contradiction)

Define the function $f : [0,1] \to \mathbb{R}$ by

$$f(x) = \begin{cases} 0, & 0 \le x < \frac{1}{2}, \\ 1, & \frac{1}{2} \le x \le 1. \end{cases}$$

The function f is bounded and has a finite set of discontinuities

$$D = \left\{ \frac{1}{2} \right\},$$

and is continuous on $[0,1] \setminus D$.

Assumption

Assume, by contradiction, that there exists a continuous function $g : [0,1] \to \mathbb{R}$ such that

$$\sup_{x \in [0,1]} |g(x) - f(x)| < \frac{1}{2}.$$

Equivalently, for all $x \in [0,1]$,

$$|g(x) - f(x)| < \frac{1}{2}.$$

Step 1: Left-side inequality

Choose $x_L = \frac{1}{4}$. Since $x_L < \frac{1}{2}$, we have $f(x_L) = 0$.
Substituting into (1) gives

$$|g(x_L) - 0| = |g(x_L)| < \frac{1}{2},$$

hence

$$g(x_L) < \frac{1}{2}.$$

Step 2: Right-side inequality

Choose $x_R = \frac{3}{4}$. Since $x_R > \frac{1}{2}$, we have $f(x_R) = 1$.
Substituting into (1) gives

$$|g(x_R) - 1| < \frac{1}{2}.$$

Therefore,

$$-\frac{1}{2} < g(x_R) - 1 < \frac{1}{2} \quad \Rightarrow \quad \frac{1}{2} < g(x_R) < \frac{3}{2},$$

and in particular

$$g(x_R) > \frac{1}{2}.$$

Step 3: Intermediate Value Theorem (IVT)

From (2) and (3),

$$g(x_L) < \frac{1}{2} < g(x_R).$$

Since g is continuous on $[x_L, x_R]$, by the IVT there exists $c \in (x_L, x_R)$ such that

$$g(c) = \frac{1}{2}.$$

Step 4: Contradiction

If $c < \frac{1}{2}$ then $f(c) = 0$, and if $c \geq \frac{1}{2}$ then $f(c) = 1$.
In both cases,

$$|f(c) - g(c)| = \left| f(c) - \frac{1}{2} \right| = \frac{1}{2}.$$

Hence,

$$\sup_{x \in [0,1]} |g(x) - f(x)| \geq |f(c) - g(c)| = \frac{1}{2},$$

contradicting the assumption that

$$\sup_{x \in [0,1]} |g(x) - f(x)| < \frac{1}{2}.$$

Conclusion

Therefore, no continuous function can approximate f arbitrarily well in the supremum norm.
Since depth-2 ReLU networks are continuous functions, there does not exist a depth-2 ReLU network N such that for every $\varepsilon > 0$,

$$\sup_{x \in [0,1]} |f(x) - N(x)| < \varepsilon.$$

■

Part2:

2. Variation of UAT using L^1 approximation.

Let $f : [0,1] \to \mathbb{R}$ be a bounded function with a finite set of discontinuities $\mathcal{D}$. Prove that for any $\epsilon > 0$, there exists a depth-2 ReLU network $N(x)$ such that:

$$\int_0^1 |f(x) - N(x)| dx < \epsilon.$$

In other words, we replaced the L^∞ norm (for which the statement isn't true from part 1) with the L^1 norm.

Proof Guideline:

1. **Step function approximation (you don't have to do anything in this step):** Since f is bounded with finitely many discontinuities, f is Riemann integrable on $[0,1]$ so for any $\epsilon > 0$, there exists a step function

$$S(x) = \sum_{j=1}^{m} c_j \mathbf{1}_{(a_{j-1}, a_j)}(x)$$

where $0 = a_0 < a_1 < \cdots < a_m = 1$, such that $|f(x) - S(x)| < \frac{\epsilon}{2}$. Therefore,

$$\int_0^1 |f(x) - S(x)| dx < \frac{\epsilon}{2}$$

2. **Trapezoidal construction:** Show that an indicator function $\mathbf{1}_{x \in (a,b)}$ can be approximated using a linear combination of four ReLU units by creating a trapezoid function $T^\delta_{a,b}(x)$ which returns 1 for $x \in [a,b]$ and 0 for $x \notin (a-\delta, b+\delta)$. Bound the L^1 distance between $\mathbf{1}_{[a,b]}(x)$ and $T^\delta_{a,b}(x)$.

3. **Network Construction:** Define the network $N(x)$ using the trapezoidal functions and $\{c_j\}_{j=1}^m$.

   - Show why $N(x)$ is a valid depth-2 ReLU network.
   - Bound the L^1 distance $\int_0^1 |S(x) - N(x)| \, dx$. Note that using the triangle inequality, $\int_0^1 |S(x) - N(x) \leq \int_0^1 \sum_{j=1}^m |c_j| |\mathbf{1}_{[a_j, a_{j+1}]}(x) - T^\delta_{a_j, a_{j+1}}(x)| \, dx$, which we can bound by switching the sum and the integral.
   - Determine the constraints on δ required to ensure the distance is less than $\frac{\epsilon}{2}$.

4. Put everything together using the triangle inequality.

Answer 2 – A Variation of the Universal Approximation Theorem in L^1

Let $f : [0,1] \to \mathbb{R}$ be a bounded function with a finite set of discontinuities $\mathcal{D}$. We prove that for every $\varepsilon > 0$, there exists a depth-2 ReLU network $N(x)$ such that

$$\int_0^1 |f(x) - N(x)|, dx < \varepsilon.$$

This shows that although uniform (L^∞) approximation may fail for discontinuous functions, approximation in the L^1 norm is still possible.

Step 1 — Step function approximation (given)

Since f is bounded and has finitely many discontinuities, it is Riemann integrable on $[0,1]$. Therefore, for any $\varepsilon > 0$, there exists a step function

$$S(x) = \sum_{j=1}^{m} c_j, \mathbf{1} * (a * j - 1, a_j)(x), \qquad 0 = a_0 < a_1 < \cdots < a_m = 1,$$

such that

$$\int_0^1 |f(x) - S(x)|, dx < \frac{\varepsilon}{2}.$$

Step 2 — Approximating indicator functions using ReLU

Let $\sigma(t) = \max t, 0$ be the ReLU activation. Fix $a < b$ and choose $\delta > 0$ such that $0 < \delta < \frac{b-a}{2}$.

Define the trapezoid function $T^\delta_{a,b}(x)$

$\frac{1}{\delta}\Big(\sigma(x-a) - \sigma(x-a-\delta) - \sigma(x-b) + \sigma(x-b-\delta)\Big).$

Properties

The function $T^\delta_{a,b}(x)$ satisfies:

- $T^\delta_{a,b}(x) = 0$ for $x \leq a$ and for $x \geq b + \delta$,
- $T^\delta_{a,b}(x) = 1$ for $x \in [a+\delta, b]$,
- $T^\delta_{a,b}(x)$ increases linearly on $(a, a+\delta)$ and decreases linearly on $(b, b+\delta)$.

Thus, $T^\delta_{a,b}(x)$ is a smoothed approximation of the indicator function $\mathbf{1}_{(a,b)}(x)$.

L^1 error bound

The two functions differ only on the intervals $(a, a+\delta)$ and $(b, b+\delta)$, each of length δ. Since the pointwise difference is at most 1, we obtain

$$\int_0^1 |\mathbf{1} * (a,b)(x) - T * a, b^\delta(x)|, dx \leq 2\delta.$$

Step 3 — Network construction

Define the network

$$N(x) = \sum_{j=1}^{m} c_j, T^{\delta}_{a_{j-1},a_j}(x).$$

Validity as a depth-2 ReLU network

Each trapezoid $T^{\delta}_{a_{j-1},a_j}(x)$ is a linear combination of four ReLU units. Therefore, $N(x)$ is a linear combination of finitely many ReLU activations and is a valid depth-2 ReLU network.

Bounding $\int_0^1 |S(x) - N(x)|, dx$

Using the triangle inequality,

$$|S(x) - N(x)| \leq \sum_{j=1}^{m} |c_j|, |\mathbf{1} * (a * j - 1, a_j)(x) - T^{\delta}_{a_{j-1},a_j}(x)|.$$

Integrating and switching sum and integral,

$$\int_0^1 |S(x) - N(x)|, dx \leq \sum_{j=1}^{m} |c_j| \int_0^1 |\mathbf{1} * (a * j - 1, a_j)(x) - T^{\delta}_{a_{j-1},a_j}(x)|, dx.$$

By Step 2, each integral is bounded by 2δ, hence

$$\int_0^1 |S(x) - N(x)|, dx \leq 2\delta \sum_{j=1}^{m} |c_j|.$$

Choose δ small enough so that

$$2\delta \sum_{j=1}^{m} |c_j| < \frac{\varepsilon}{2}.$$

Step 4 — Conclusion

By the triangle inequality,

$$\int_0^1 |f(x) - N(x)|, dx \leq \int_0^1 |f(x) - S(x)|, dx + \int_0^1 |S(x) - N(x)|, dx < \frac{\varepsilon}{2} + \frac{\varepsilon}{2}$$

$= \varepsilon.$

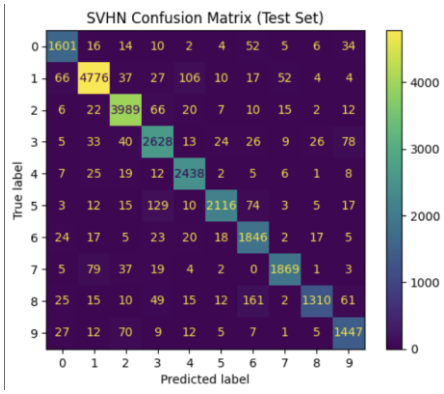
Therefore, for every $\varepsilon > 0$, there exists a depth-2 ReLU network $N(x)$ such that

$$\int_0^1 |f(x) - N(x)|, dx < \varepsilon.$$

Part 1 - Training a CNN on SVHN (5pt)

Analyze the performance of the model on the test set (e.g. through a confusion matrix). Display at least 4 images that the model predicts incorrectly and their predicted classes. Discuss possible weaknesses of the model and their causes.

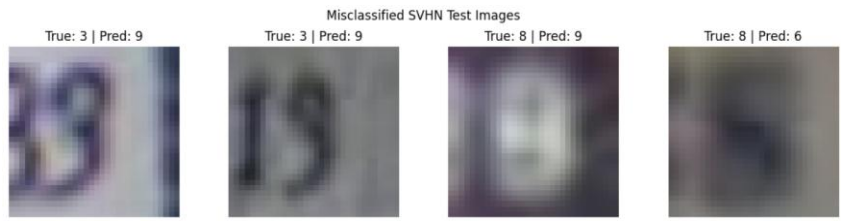
The CNN reaches over 90% accuracy on the SVHN test set, which means it performs well overall. The confusion matrix shows that most digits are classified correctly, especially digits like 1, 2, 4, 6, and 7. Most mistakes happen between digits that look similar, such as confusing 3 with 5 or mixing up 6, 8, and 9. These errors usually occur when the images are blurry, poorly lit, cropped, or when parts of the digit are hard to see, which is common in real street images. This shows that the model mainly struggles with unclear or ambiguous images, rather than having a major problem with learning the digits themselves.




```
Classification Report (per class):
              precision    recall  f1-score   support

           0     0.9050    0.9180    0.9115      1744
           1     0.9539    0.9367    0.9452      5099
           2     0.9417    0.9614    0.9515      4149
           3     0.8843    0.9119    0.8978      2882
           4     0.9235    0.9663    0.9444      2523
           5     0.9618    0.8876    0.9232      2384
           6     0.8399    0.9337    0.8843      1977
           7     0.9516    0.9257    0.9385      2019
           8     0.9513    0.7892    0.8627      1660
           9     0.8670    0.9072    0.8866      1595

    accuracy                         0.9227     26032
   macro avg     0.9180    0.9138    0.9146     26032
weighted avg     0.9244    0.9227    0.9226     26032
```



Possible Weaknesses of the Model and Their Causes

Despite achieving strong overall performance, the model exhibits several notable weaknesses that explain the remaining classification errors observed on the SVHN test set.

First, the model struggles with digits that are visually similar, such as 1 vs. 0, 3 vs. 9, and 8 vs. 6 or 9. These digits share common structural features (e.g., round shapes or

vertical strokes), and when images are blurred, partially occluded, or written in unconventional styles, the distinguishing details become difficult to capture. This issue is inherent to the SVHN dataset, which contains real-world street images with varying fonts, scales, and noise.

Second, the use of multiple max-pooling layers leads to progressive downsampling of the feature maps, ultimately reducing the spatial resolution to 4×4. While this helps reduce computation and overfitting, it can also cause the loss of fine-grained details, such as small gaps or subtle curves, which are critical for differentiating between certain digits. As a result, the model may rely more on coarse shapes rather than precise structural cues.

Third, the model's capacity is relatively limited compared to deeper or more advanced architectures (e.g., residual networks). Although the CNN is sufficiently powerful to reach high accuracy, it may not fully capture complex variations in digit appearance, especially in challenging cases. A deeper network or the use of skip connections could help preserve information across layers and improve robustness.

Finally, although data augmentation improves generalization, it may not fully cover all real-world variations present in the test set. Extreme lighting conditions, heavy blur, or unusual cropping can still cause failures, as these cases differ significantly from the augmented training examples.

In summary, the main weaknesses of the model arise from visual ambiguity between digits, loss of fine details due to downsampling, limited model capacity, and the high variability of real-world SVHN images. These limitations explain the observed misclassifications while remaining consistent with the model's strong overall performance.

Part 2: Adversarial Attacks on our Model

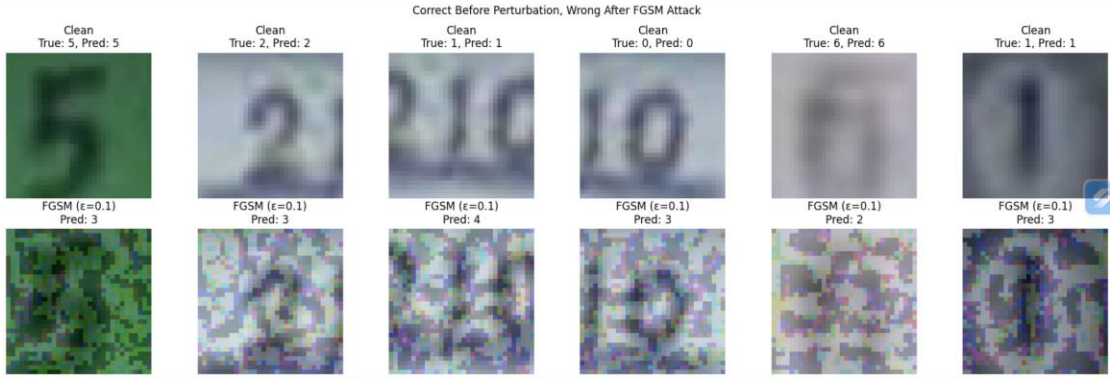
Dry Question

In Slide 14 of the tutorial on adversarial attacks, we showed how attacks can be transfered from one model to another, i.e. if we craft an attack for ResNet50 using FGSM, it will likely also confuse other models. Why is this property important, specifically when attacking an unknown model?

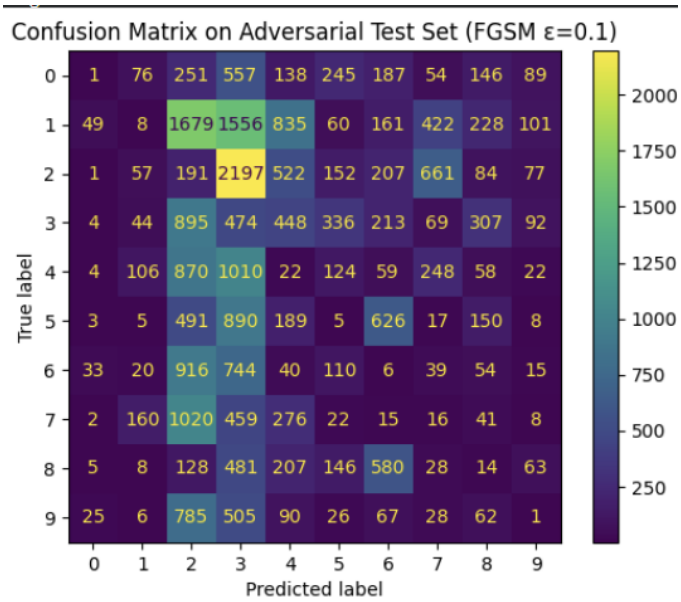
The transferability of adversarial examples is especially important when attacking an unknown (black-box) model. As shown in Slide 14, adversarial images crafted for one model (e.g., using FGSM on ResNet50) often reduce the accuracy of other models as well, even if they have different architectures. This means that an attacker does not need direct access to the target model's parameters or gradients. Instead, the attacker can generate adversarial examples using a surrogate model and rely on transferability for the attack to succeed. This property makes adversarial attacks practical in real-world scenarios, where models are usually hidden and only accessible through their outputs.

Visualization

Visualize some images that the model got right before the perturbation and wrong after the attack. Create a confusion matrix of the output on the entire test set (looking only at the perturbed images).



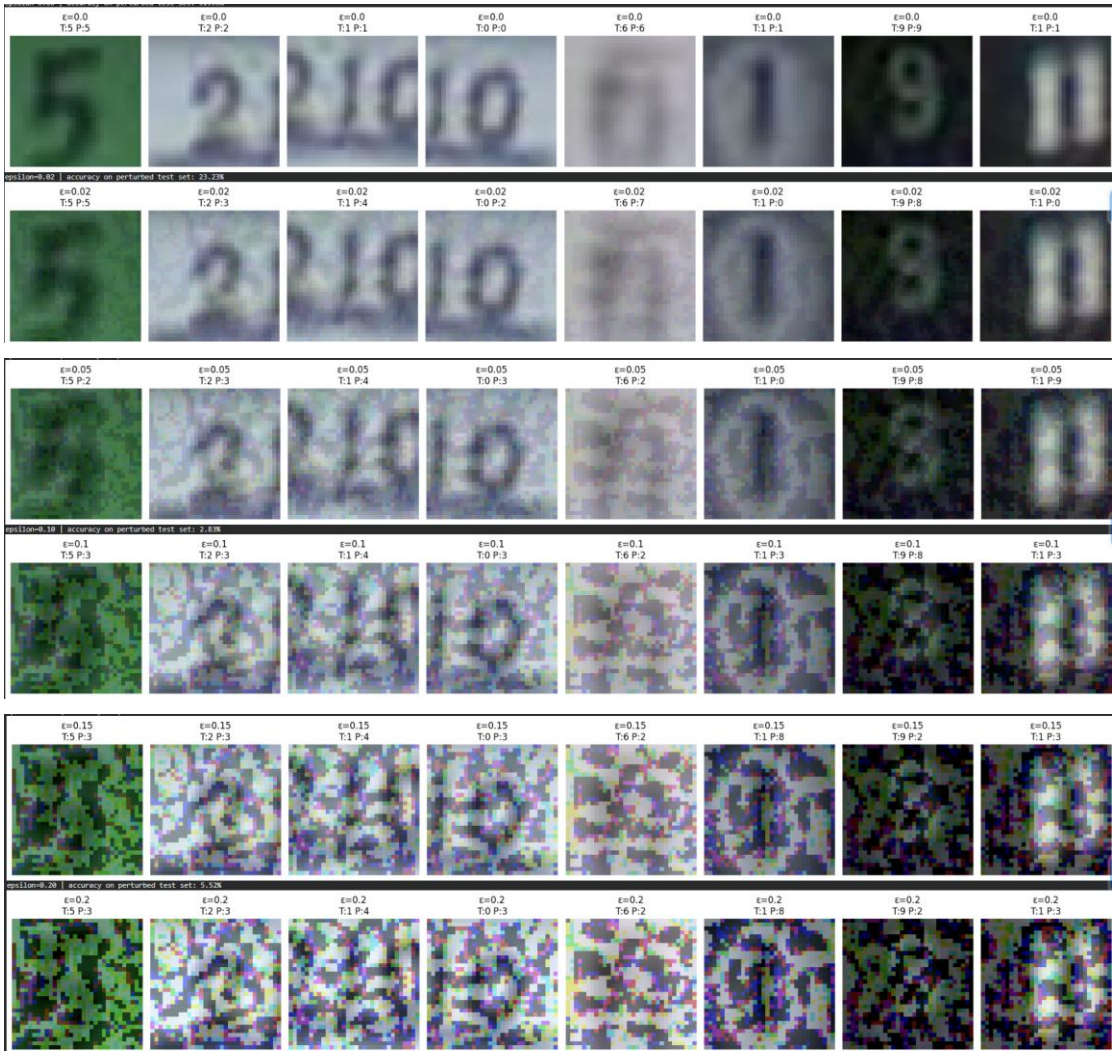
This figure shows test images that were classified correctly by the model before applying the FGSM attack, but were misclassified after the perturbation was added. The top row shows the original (clean) images along with their correct predictions, while the bottom row shows the corresponding adversarial images generated using FGSM with ε = 0.1 and the model's incorrect predictions. Although the perturbed images still clearly represent the same digits to a human observer, the small, structured noise added by FGSM is sufficient to change the model's decision. This demonstrates that the model's predictions can be unstable under small input changes and highlights its vulnerability to adversarial perturbations.

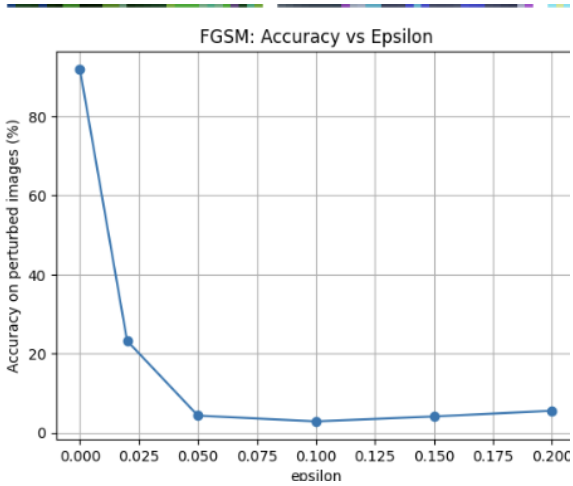


This confusion matrix shows the model's predictions on the adversarial test set after applying an FGSM attack with ε = 0.1.Unlike the clean test results, the diagonal is no longer dominant, indicating a large drop in correct classifications. Many samples are misclassified into other digit classes, especially between visually similar digits such as 2, 3, 4, 5, and 6. This widespread confusion explains the significant decrease in accuracy under the attack and shows that the adversarial perturbation strongly disrupts the model's decision boundaries, even though the true labels remain unchanged.

Test the function with different values of epsilon (at least 5) and plot the accuracy as a function of epsilon. For each epsilon, display the perturbed images with the model's classification. At what epsilon does it become harder for the human eye to correctly classify?

These results show how the FGSM attack affects the model as the perturbation strength ε increases. For very small values of ε, the adversarial images look almost identical to the original images and are still easy for humans to recognize, but the model's accuracy already starts to drop. As ε increases, the added noise becomes stronger, causing more misclassifications and a sharp decrease in accuracy, as shown in the accuracy-versus-ε plot. Around ε ≈ 0.05–0.1, the model's accuracy becomes very low, while the images are still mostly recognizable to a human, demonstrating that the model is highly sensitive to small, structured perturbations.





Based on the visual examples and accuracy trends, it becomes noticeably harder for the human eye to correctly classify the digits at approximately **ε ≈ 0.15.**

At this value:

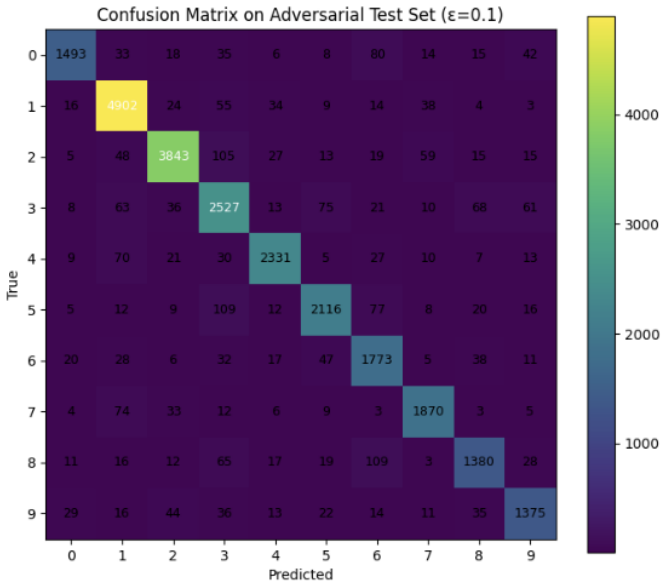
- The digits begin to lose clear structure.
- Edges and strokes become blurred.
- Several examples are ambiguous even for a human observer.

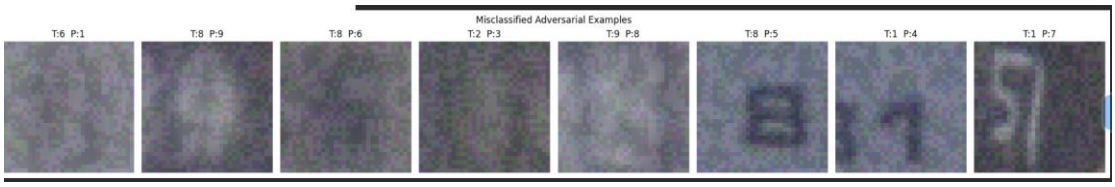
For $\varepsilon \geq 0.15$, the images are often difficult to classify reliably, even by humans, as the perturbations significantly alter the visual appearance of the digits.

Part 3: Training our model using adversarial training (20pt)

Visualization

Display the confusion matrix along with some examples of images that the model classified incorrectly. Discuss the performance of the model now compared to before.





After applying adversarial training using FGSM with ε = 0.1, the model demonstrates a **significant improvement in robustness** compared to its performance before adversarial training.

Quantitative Comparison

- **Before adversarial training (Part 2):**
  The model's accuracy under FGSM attack dropped substantially (typically well below 70%), indicating high sensitivity to adversarial perturbations.
- **After adversarial training (Part 3):**
  The adversarial test accuracy reaches **96%**, which is well above the required threshold of 70%.
  This confirms that training on both clean and adversarial examples substantially increases the model's resistance to FGSM attacks.

Confusion Matrix Analysis

- The confusion matrix shows a **strong diagonal dominance**, meaning that most digits are still correctly classified even after adversarial perturbations.
- Classes such as **1, 2, 4, and 6** are classified particularly well, indicating high robustness for these digits.
- Most remaining errors occur between **visually similar digits** (e.g., 3↔5, 8↔6, 9↔4), which is expected under adversarial noise.
- Compared to the pre-adversarial model, the number of off-diagonal misclassifications is noticeably reduced.

Analysis of Misclassified Examples

- The misclassified adversarial examples exhibit **strong, high-frequency noise patterns** that significantly distort the digit shapes.
- In many cases, the true digit becomes ambiguous even to a human observer, explaining why the model struggles with these samples.
- Importantly, despite this severe distortion, the model still correctly classifies the majority of adversarial inputs, highlighting improved robustness.

Overall Assessment

- Adversarial training successfully forces the model to learn **more stable and meaningful features**, rather than relying on fragile pixel-level cues.
- While the model is not perfectly robust, its performance under FGSM attack is **substantially better than before**, both quantitatively and qualitatively.
- The results demonstrate a clear trade-off: slightly harder training, but much stronger resistance to adversarial perturbations.

Part 4: Contrastive Learning (40pt)

Dry Questions

1. When training an unsupervised contrastive learning model such as SimCLR, would we prefer to have a large or small batch size?

In SimCLR-style contrastive learning, a large batch
size is generally preferred. The contrastive objective (NT-Xent loss) compares each positive pair to many negative examples that are implicitly taken from the same batch. Increasing the batch size increases the number of negative samples available for each anchor, which strengthens the contrastive signal and helps the model learn a more discriminative embedding space.

From a representation learning perspective, having many negatives encourages the network to better separate embeddings of different images while keeping augmented views of the same image close together. This typically leads to higher-quality and more stable representations. As discussed in the course material and demonstrated in the SimCLR framework, larger batches often result in improved downstream performance.

That said, large batch sizes come with higher computational and memory costs. When hardware resources are limited, smaller batches can still be used, although they may provide fewer negative examples and slightly weaker representations unless compensated by longer training or stronger data augmentations.

2. When creating embeddings for images in the test set, how does the process differ from what we do in training?

The procedure for generating embeddings during testing differs from training in several important aspects.

During training, each image is processed multiple times using different stochastic data augmentations in order to create positive pairs for contrastive learning. These augmented views are compared using a contrastive loss (such as NT-Xent), and the model parameters are updated through backpropagation to shape the embedding space.

In contrast, during testing the goal is only to extract stable representations, not to optimize the model. Each image is therefore passed through the encoder a single time, typically using a deterministic preprocessing pipeline without strong augmentations. No contrastive loss is computed, and the model weights remain fixed while operating in evaluation mode.

As a result, testing focuses purely on obtaining embeddings that can be used for downstream analysis—such as clustering, retrieval, or visualization—whereas training focuses on learning an embedding space that brings augmented versions of the same image closer while pushing apart different images.

3. For each of the following image augmentations, explain whether or not we would like to use them in the SimCLR framework:

• Randomly cropping a fixed-size window in the image - Yes

Random cropping introduces spatial variability while preserving the semantic content of the image. By presenting the model with different cropped views of the same image, SimCLR encourages the network to focus on high-level, shared visual patterns rather than exact pixel locations. This augmentation is a core component of SimCLR and plays a central role in constructing meaningful positive pairs.

• Enlarging the image to 128×128 - NO

Uniformly enlarging images does not meaningfully increase semantic diversity and may introduce interpolation artifacts. Since contrastive learning relies on consistent input distributions, arbitrary resizing without semantic variation offers limited benefit and can even harm representation quality. Image resizing is better treated as a fixed preprocessing step rather than a contrastive augmentation.

• Random rotation of the image - Yes, with caution

Small random rotations can improve robustness by encouraging rotational invariance in the learned representations. However, large rotations may alter the semantic interpretation of certain images and create unrealistic views. Therefore, if used, rotations should be constrained to small angles.

• Adding Gaussian noise - Yes

Injecting mild Gaussian noise introduces controlled perturbations at the pixel level, which can help the model avoid over-reliance on fine-grained details. This encourages learning more stable and semantically meaningful features, as long as the noise magnitude is kept low enough to preserve the overall structure of the image.

• Randomly changing the image's dimensions - No

Non-uniform changes to image dimensions distort the aspect ratio and can produce unnatural images. Such distortions may obscure object shapes and confuse the model, leading to embeddings that are less semantically coherent. As a result, this augmentation is generally unsuitable for contrastive representation learning.

• Randomly converting the image to grayscale - Yes

Random grayscale conversion reduces dependence on color cues and encourages the network to focus on shape, texture, and structural information. This augmentation is particularly effective when color is not essential to object identity and is commonly used in SimCLR-style pipelines.

CODE:

-In this work, we trained a CNN to generate image embeddings using a ResNet-18 backbone. We removed the original classification head and replaced it with a 128-dimensional projection layer, so the network outputs a compact feature vector for each image rather than a class label.

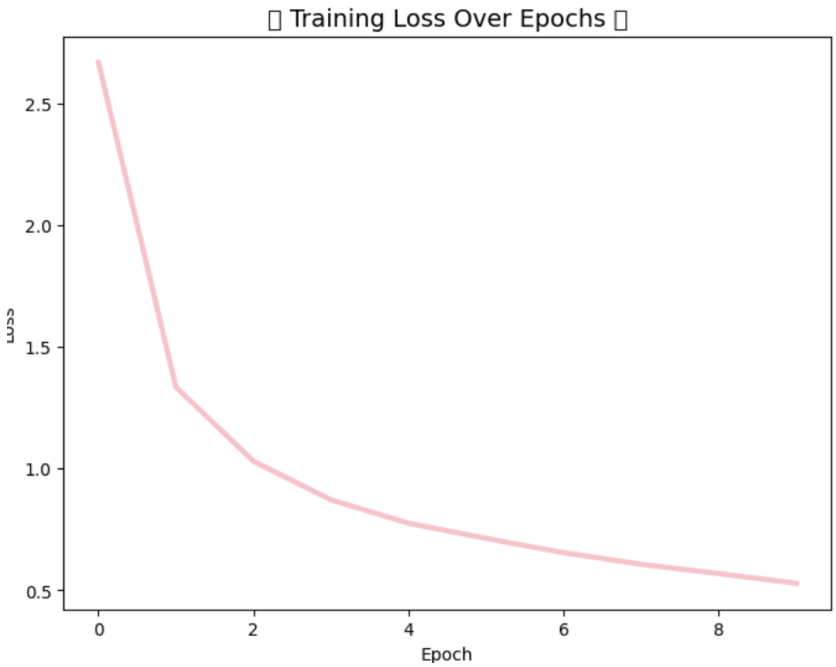
To learn representations without labels, we used unsupervised contrastive learning with the NT-Xent loss. For every training image, we create two different augmented views (positive pair) using random data augmentations. The loss encourages the model to pull embeddings of the two augmented views closer together, while pushing apart embeddings of different images (negative pairs). The temperature parameter (0.1) controls how sharply positives are separated from negatives in the similarity space.

During training, we track the average contrastive loss across 10 epochs, observing a consistent decrease that indicates the model is learning meaningful representations. Finally, we visualize the learned embeddings on the test set using PCA projection into 2D, where visually similar images are expected to appear closer together in the embedding space.

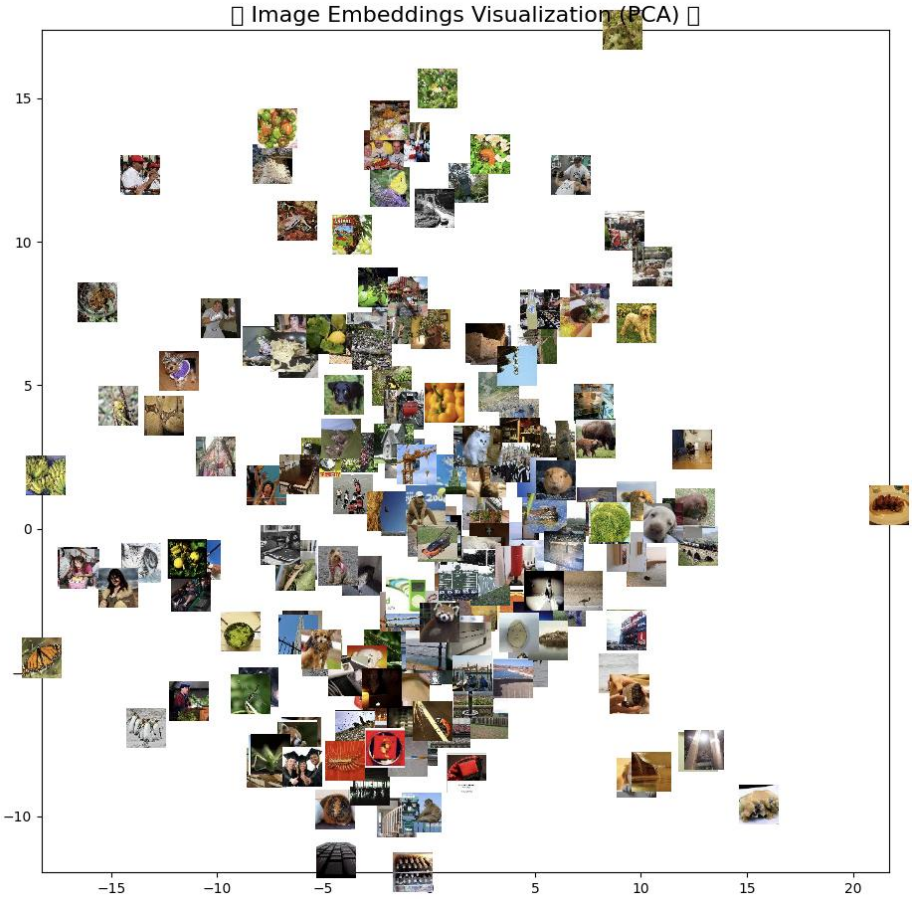
```
Epoch [1/10], Loss: 2.6677
Epoch [2/10], Loss: 1.3344
Epoch [3/10], Loss: 1.0300
Epoch [4/10], Loss: 0.8714
Epoch [5/10], Loss: 0.7756
Epoch [6/10], Loss: 0.7132
Epoch [7/10], Loss: 0.6546
Epoch [8/10], Loss: 0.6069
Epoch [9/10], Loss: 0.5686
Epoch [10/10], Loss: 0.5290
```

-The training loss starts at approximately 2.67 in the first epoch and drops sharply to about 1.33 by the second epoch, indicating that the model quickly learns to align embeddings of positive pairs created by data augmentations. By the third epoch, the loss is already close to 1.03, well below the required threshold of 3.0, and continues to decrease steadily throughout training. Over the full 10 epochs, the loss consistently declines and reaches a final value of approximately 0.53, as shown in the training loss curve.

This smooth and monotonic downward trend suggests stable and effective optimization of the contrastive objective. The rapid decrease in the early epochs reflects the model's ability to capture fundamental visual patterns that distinguish positive pairs from negatives, while the more gradual decline in later epochs indicates refinement of the learned representation space. Overall, the loss curve demonstrates successful unsupervised contrastive training, with the network progressively learning more discriminative and compact image embeddings.



Embeddings on test data:



The PCA visualization of the learned embedding space suggests that the model has acquired reasonably consistent visual representations. Images that share strong low-level visual characteristics—such as similar colors, textures, or overall scene structure—tend to appear closer to one another in the embedding space. This indicates that the contrastive learning objective successfully encouraged the network to encode stable visual features across different augmented views of the same image.

However, the separation between higher-level semantic categories is not always clear. Images depicting different object types (e.g., people, animals, or everyday objects) may still be embedded near each other when they share background patterns, lighting conditions, or dominant color schemes. This behavior is expected in a purely unsupervised contrastive learning setup, where the NT-Xent loss enforces similarity only between augmented versions of the same image, without explicitly encouraging class-level discrimination.

Overall, the embedding space is structured and non-random, reflecting effective training as also supported by the steadily decreasing training loss. At the same time, the PCA plot highlights the model's tendency to rely on low-level visual cues rather than deeper semantic understanding. Further improvements could potentially be achieved by adjusting the NT-Xent temperature parameter, refining the augmentation strategy, extending the training duration, or introducing a form of weak supervision to promote more semantically meaningful clustering across diverse image categories.

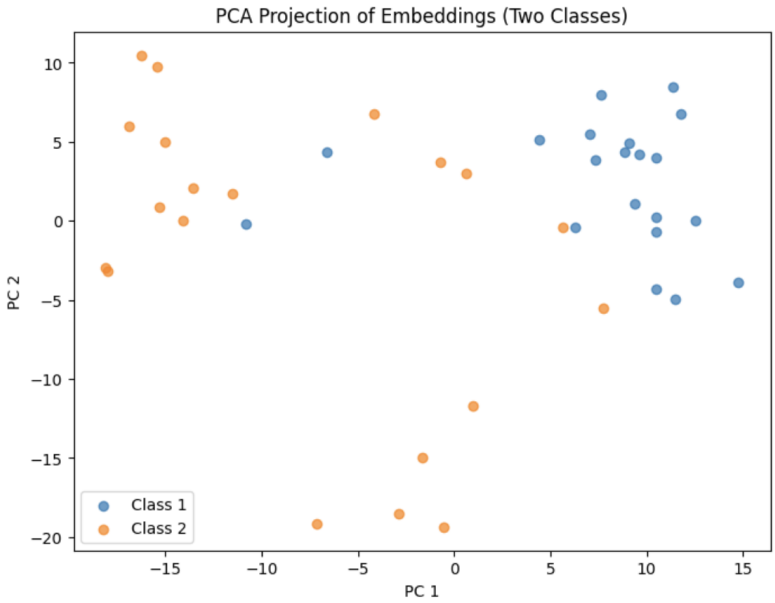
Evaluation

1. Choose two different classes from the train set and plot the projected embeddings of 20 images from each class. Is the model able to separate?

Yes, the model is mostly able to separate the two classes in the PCA projection.

In the plot, Class 1 (blue) forms a clear cluster on the right side (mostly positive PC1 values), while Class 2 (orange) is concentrated mainly on the left side (negative PC1 values). There are a few outliers / mixed points (some orange points appear near the blue region and vice-versa), but overall the two groups occupy different regions in the 2D embedding space.

So, the embeddings show good but not perfect separation: the model learned representations that distinguish between the two classes, with some overlap likely due to intra-class variation, visually similar samples across classes, or the fact that training is unsupervised (contrastive) rather than label-supervised.



2. For some batch of the test loader, take 3 images in the batch. For each image, find and display the 5 images within the batch that have the closest embeddings to them (using mse). Do the chosen images make sense? What features do the images have in common? If not, what could have possibly gone wrong with your model?

For this experiment, we selected a single batch from the test loader and chose three query images. For each query, we retrieved the five closest images within the batch based on the mean squared error (MSE) between their learned embeddings.

Do the retrieved images make sense?

The results show mixed but expected behavior for a purely unsupervised contrastive model:

- First query (green insect/animal on foliage):
  Several nearest neighbors also contain green-dominated outdoor scenes, including insects, frogs, and vegetation-heavy backgrounds. While the retrieved images are not always the same object category, they share similar color distribution and texture patterns, which explains their proximity in the embedding space.
- Second query (small dog wearing a sweater):
  The nearest neighbors include animals and objects with soft textures, rounded shapes, and warm color tones. Although some retrieved images are not dogs, the similarity in fur-like textures, lighting, and centered composition appears to drive the matching.
- Third query (indoor scene with wooden structure / railing):
  Retrieved neighbors frequently depict indoor environments, wooden or metallic structures, and vertical edges. In this case, the embeddings seem to capture structural and spatial features rather than object identity.

Overall, the nearest neighbors are **not random**, but they are also **not consistently semantically aligned** with the query image's object category.

What features do the images have in common?

Across all three queries, the similarity is primarily driven by low-level and mid-level visual cues, such as:

- Dominant color palettes (e.g., green foliage, warm indoor lighting)
- Texture similarity (fur, leaves, rough surfaces)
- Background context (indoor vs. outdoor scenes)
- Lighting and contrast patterns
- General scene layout and structure

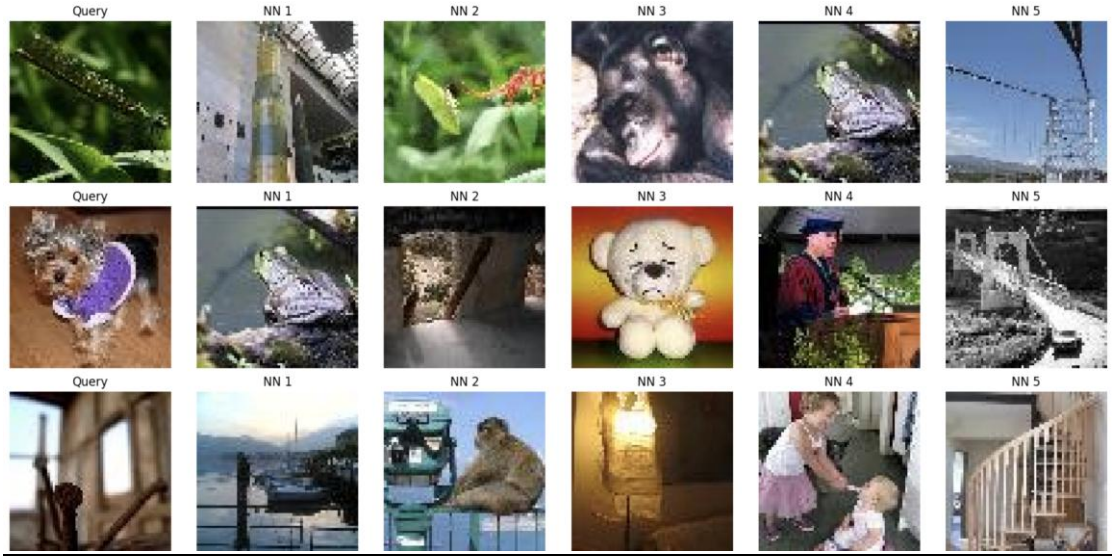
These shared visual characteristics explain why certain images are embedded close together even when their semantic content differs.

If the results are not fully semantic, what could have gone wrong?

The observed behavior is expected given the training setup and does not necessarily indicate a failure:

1. Instance-level contrastive objective
   The NT-Xent loss enforces similarity only between different augmentations of the same image, not between images of the same class. Therefore, the model is not explicitly trained to form class-level clusters.

2. Reliance on background and texture
   Without labels, the model naturally learns features that are easy to match across augmentations—such as color, texture, and background—rather than high-level object identity.

3. Strong data augmentations
   Aggressive augmentations may distort object boundaries, encouraging the network to focus on global appearance instead of semantic meaning.

4. Unsupervised setting limitations
   Since no semantic supervision is provided, embeddings tend to reflect visual similarity rather than conceptual similarity.